CO4 – AT1

Data Interpretation Assessment

Name : S . Adhikesavan Reg no : 192521172

Course : Computer Vision Course Code : ITA0503

Q1. Real-Time Detection System

The table below lists three widely used detection techniques along with their reported accuracy and qualitative speed rating.

Method	Accuracy	Speed
Viola-Jones	78%	Fast
YOLO	92%	Moderate
Deep Learning (CNN-based)	95%	Slow

Fig 1.1 — Accuracy vs Speed: Viola-Jones, YOLO, Deep Learning

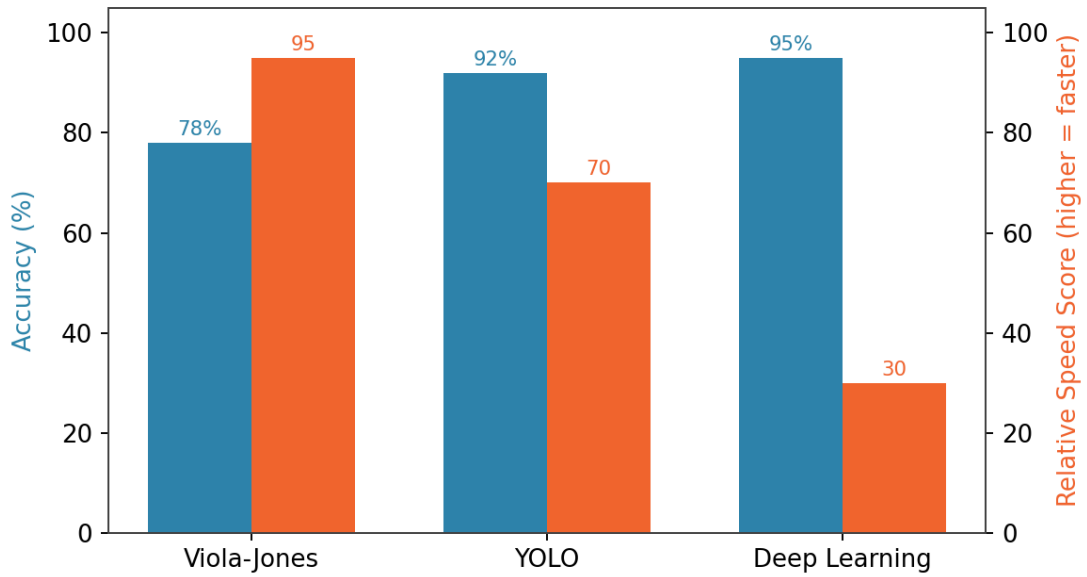

Fig 1.1 — Accuracy climbs from Viola-Jones to Deep Learning while processing speed falls in the opposite direction.

Comparison

The three methods sit along a clear accuracy-speed trade-off curve. Viola-Jones relies on Haar-like features and a cascade of simple classifiers, which makes it computationally light but limits it to frontal, well-lit faces, so its accuracy tops out around 78%. YOLO reframes detection as a single regression problem over a grid, which lets it run in a single forward pass and reach 92% accuracy at moderate speed. Deep learning pipelines (deeper CNNs, multi-stage detectors, attention-based backbones) push accuracy to 95% but do so through many more layers and parameters, which slows inference considerably.

Evaluation and Recommendation

For a surveillance system operating under strict latency constraints, YOLO is the most suitable choice. Viola-Jones is fast enough but its 78% accuracy is too low for a security-critical application — missed detections directly translate into blind spots. Deep learning offers the highest accuracy but its slow inference makes it unsuitable for live video streams where frames must be processed in real time; a backlog of frames would build up and defeat the purpose of live surveillance. YOLO's 92% accuracy is close enough to the deep learning ceiling while its moderate speed is compatible with real-time constraints, especially when run on GPU-accelerated or edge-AI hardware (e.g., NVIDIA Jetson, Coral TPU).

Justification Considering Deployment Conditions

Surveillance cameras typically stream continuous footage at 15–30 frames per second, and any detector deployed on such a feed must keep pace with the incoming frame rate or frames will be dropped. YOLO's single-pass architecture is specifically designed for this kind of throughput, and its accuracy is high enough to reliably flag intruders, unattended objects, or unusual activity. Deep learning models remain valuable for offline forensic analysis of recorded footage, where latency is not a concern and the extra 3% accuracy matters more. Viola-Jones can still be justified in low-power, cost-constrained edge devices (e.g., simple entry-point face counters) where 78% accuracy is acceptable and hardware budgets are minimal.

Q2. Robust Object Detection

The following table reports detection accuracy of the same three methods under normal, occluded and low-light viewing conditions.

Method	Normal	Occlusion	Low Light
Viola-Jones	80%	60%	55%
YOLO	90%	75%	70%
Deep Learning	95%	85%	80%

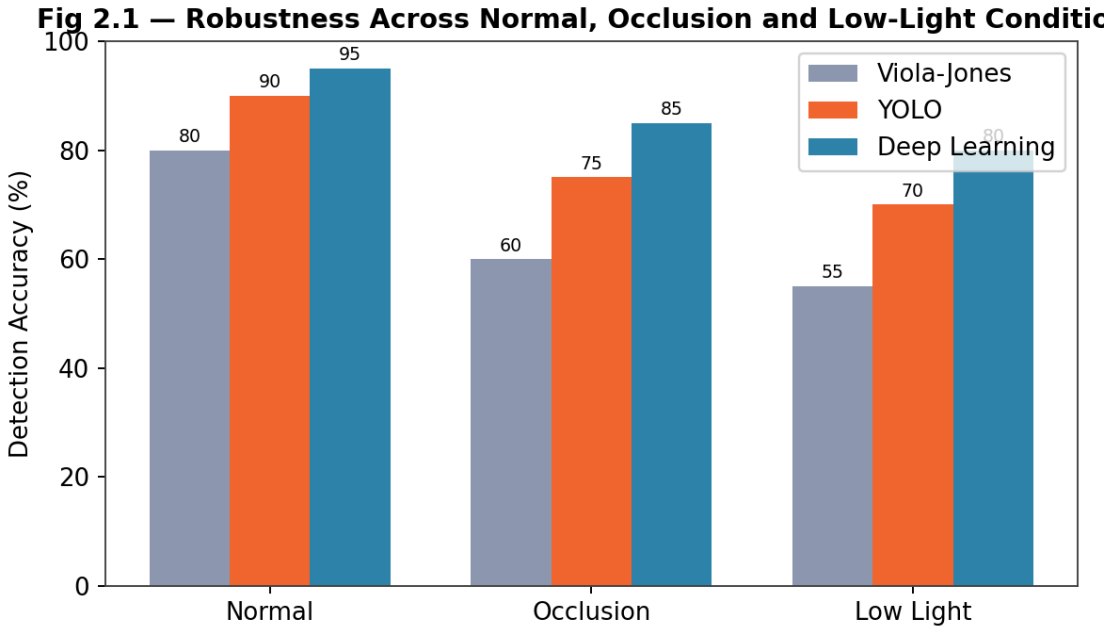

Fig 2.1 — Deep Learning degrades least across all three conditions; Viola-Jones degrades most under occlusion and low light.

Comparison

All three methods lose accuracy as conditions worsen, but the size of the drop differs sharply. Viola-Jones falls from 80% to 55%, a drop of 25 percentage points, because Haar-cascade features depend on consistent edge and intensity patterns that occlusion and poor lighting directly disrupt. YOLO falls from 90% to 70%, a 20-point drop, since its learned convolutional features generalize better than hand-crafted ones but still rely on visible spatial context. Deep learning degrades the least, from 95% to 80%, a 15-point drop, because deeper networks trained on large and diverse datasets learn more abstract, illumination- and occlusion-invariant representations.

Evaluation and Trade-offs

Deep learning provides the best overall performance in every condition and the smallest relative degradation, making it the most robust option in absolute terms. The trade-off is computational cost: the same depth and representational richness that make it robust also make it slower and more resource-hungry, which reduces its suitability where real-time response is essential (as established in Q1). YOLO offers a middle ground — a smaller robustness margin than deep learning but still a comfortable lead over Viola-Jones — while remaining fast enough for live operation. Viola-Jones should only be considered where lighting is controlled and occlusion is unlikely, such as a fixed indoor badge-scanning camera.

Conclusion

If robustness under adverse conditions is the primary design criterion and latency can be relaxed (e.g., a back-end analytics server processing recorded footage), Deep Learning is the correct choice. If the system must remain robust and still operate in real time, YOLO offers the best practical balance, since its accuracy under occlusion and low light (75% and 70%) remains usable while its speed stays within real-time limits.

Q3. Optical Flow Analysis

Two optical flow estimation strategies are compared: Method A, which produces stable motion vectors but misses small motions, and Method B, which detects fine motion but produces noisy vectors.

Fig 3.1 — Optical Flow Vector Fields: Method A vs Method B

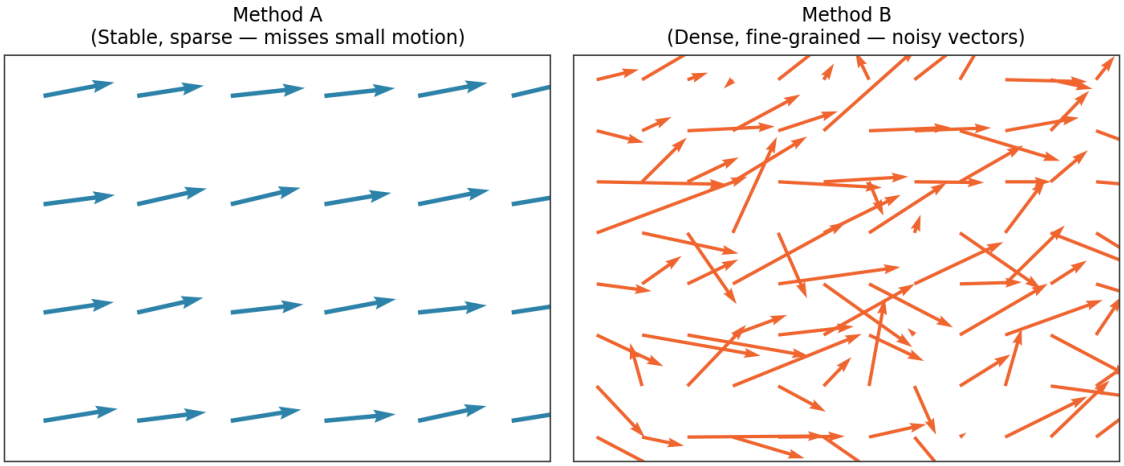

Fig 3.1 — Method A yields a smooth, uniform vector field (large-scale motion only); Method B yields a denser field that captures fine detail but with scattered, noisy directions.

Comparison

Method A behaves like a sparse, locally-averaged flow estimator — comparable to a pyramidal Lucas-Kanade approach — where motion is computed over a neighbourhood of pixels. Averaging over a window suppresses noise and produces smooth, stable vectors, but it also smooths away small or subtle motions that fall below the averaging window's sensitivity. Method B behaves like a dense, per-pixel flow estimator — comparable to a Horn-Schunck-style or unconstrained per-pixel approach — which is sensitive enough to pick up fine-grained motion but, lacking strong regularization, is also sensitive to illumination changes and sensor noise, producing scattered vectors.

Evaluation for Dynamic Scenes

In a genuinely dynamic scene — multiple objects moving at different speeds, partial occlusions, background clutter — neither method alone is ideal. Method A would under-report small but meaningful motions (e.g., a hand gesture, a distant pedestrian, an object just starting to move), which is a serious limitation for applications such as gesture recognition or early motion alerting. Method B would capture those fine motions but the noise could generate false positives or unstable tracks, which is problematic for applications like vehicle trajectory estimation where a smooth, reliable path is required.

Justification and Recommendation

For motion tracking in dynamic scenes, Method B is generally more suitable, provided its noise is controlled with a post-processing step such as a Kalman filter, median filtering, or temporal smoothing across frames. This is because missing a genuine motion (Method A's weakness) is usually a more costly failure in tracking than a noisy-but-present detection (Method B's weakness), since noise can be filtered but a missed detection cannot be recovered after the fact. In practice, a hybrid pipeline that uses Method B for its sensitivity and layers in a smoothing/outlier-rejection stage borrowed from Method A's stability philosophy tends to give the best

accuracy-stability balance — this is exactly the reasoning behind modern optical flow networks (e.g., FlowNet, RAFT) that combine dense estimation with learned regularization.

Q4. Model Generalization and Overfitting

Two trained models are compared using their training and testing accuracy, which reveals how well each model generalizes to unseen data.

Model	Training Accuracy	Testing Accuracy	Gap
Model A	98%	70%	28%
Model B	90%	88%	2%

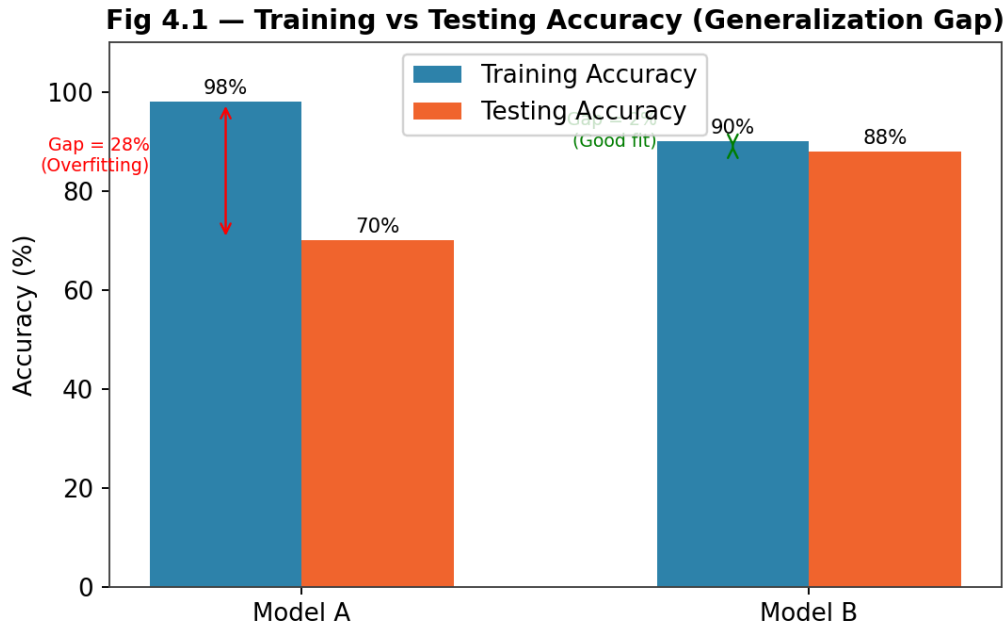

Fig 4.1 — Model A shows a large training-testing gap (overfitting); Model B's small gap indicates strong generalization.

Comparison

Model A achieves the highest training accuracy of the two (98%) but its testing accuracy collapses to 70%, leaving a 28-point gap between how well it fits the training data and how well it performs on data it has never seen. This is the textbook signature of overfitting: the model has effectively memorized noise and idiosyncrasies specific to the training set rather than learning patterns that transfer. Model B trains to a slightly lower 90% but its testing accuracy of 88% is almost identical, a gap of only 2 points, indicating that it has learned genuinely generalizable features rather than memorizing the training set.

Evaluation of Generalization Ability and Overfitting Risk

Overfitting risk is best judged by the size of the train-test gap rather than by training accuracy alone, since a model can trivially reach near-100% training accuracy by memorizing examples without learning anything useful. Model A's large gap signals high variance — it is likely too complex relative to the amount or diversity of training

data, or was trained for too many epochs without adequate regularization (dropout, weight decay, early stopping, data augmentation). Model B's small gap signals a good bias-variance balance: it may be slightly simpler or better regularized, and consequently its performance on the training set is a much more trustworthy predictor of its performance in deployment.

Recommendation for Real-World Deployment

Model B is the more reliable choice for real-world deployment. In production, a system only ever encounters new, unseen inputs — never the exact training data — so testing accuracy (or better, accuracy on a held-out validation/production-like set) is the metric that actually matters. Model A's impressive training number is misleading, and deploying it would likely result in unpredictable, degraded real-world performance close to its 70% testing accuracy, with the added risk that performance could vary unpredictably depending on how closely new inputs resemble the training distribution. Model B's smaller gap and higher testing accuracy (88% vs 70%) make its behaviour far more predictable and trustworthy once deployed.

Q5. System Performance under Constraints

Three candidate models are evaluated on accuracy, latency and relative deployment cost for a real-time application.

Model	Accuracy	Latency	Cost
Model A	88%	50 ms	Low
Model B	92%	80 ms	Moderate
Model C	95%	120 ms	High

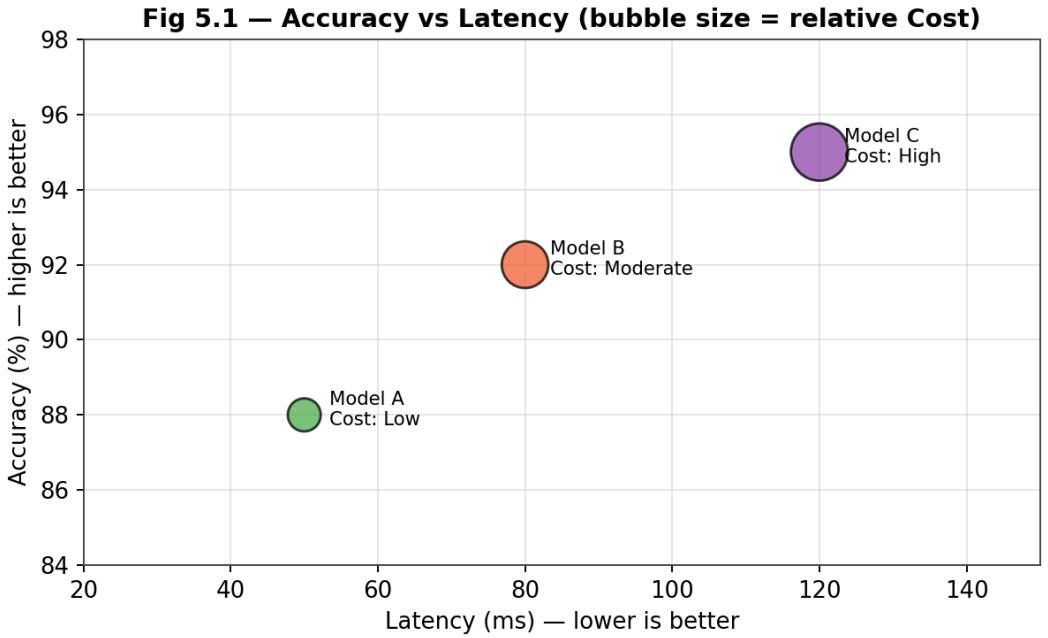

Fig 5.1 — Model B occupies the balanced middle ground between Model A's speed/low cost and Model C's accuracy/high cost.

Comparison

The three models trace a consistent pattern: each additional gain in accuracy is paid for with higher latency and higher deployment cost. Model A is the fastest and cheapest at 50 ms latency and low cost, but its 88% accuracy is the lowest of the three. Model C delivers the best accuracy at 95%, but at 120 ms latency and high cost, it is more than twice as slow as Model A and the most expensive to deploy and maintain. Model B sits in between on every axis — 92% accuracy, 80 ms latency, moderate cost.

Evaluation of Balance

For a real-time application, latency has a hard ceiling: if a model's response time exceeds the time budget available per frame or per transaction, the system cannot be considered real-time regardless of how accurate it is. Model C's 120 ms latency is the most likely to breach such a budget in a live pipeline (e.g., a video system running at 15–30 fps needs sub-70 ms per-frame processing to keep up), and its high cost compounds the problem for large-scale deployment. Model A meets latency and cost targets comfortably but sacrifices 7 percentage points of accuracy compared to Model C, which may be too large a loss for applications where errors carry real consequences (e.g., safety or security alerts).

Recommendation and Justification

Model B offers the best overall balance for a real-time application under practical deployment constraints. Its accuracy (92%) is close to Model C's ceiling while its latency (80 ms) remains within a range that most real-time pipelines can absorb, and its moderate cost is sustainable for ongoing deployment at scale. Model A remains the right choice only when latency and cost are the dominant constraints and a lower accuracy ceiling is acceptable (e.g., a low-power edge sensor performing coarse pre-filtering), while Model C is best reserved for scenarios where accuracy is paramount and either latency is relaxed or the cost of extra compute is justified by the stakes involved (e.g., offline verification of flagged events).

Overall Summary

Across all five scenarios, the same underlying principle recurs: no single method is universally 'best' — the right choice depends on which constraint (accuracy, speed, robustness, generalization, or cost) is binding for the specific deployment. YOLO and Model B repeatedly emerge as balanced, real-time-friendly options, Deep Learning and Model C represent accuracy ceilings best suited to offline or latency-tolerant scenarios, and Viola-Jones/Model A represent lightweight options for constrained or lower-stakes environments. Recognising and stating this trade-off explicitly, rather than picking a single 'winner' in isolation, is the key evaluative skill these questions are testing.